Assignment 2: Simple Search Engine using Hadoop MapReduce

1. Methodology

1.1 Data Preparation

I used PySpark to extract documents from the Wikipedia parquet file. The `prepare_data.py` script reads the parquet, samples around 1200 rows, and writes each article as a separate `.txt` file. The filename is built from the doc id and title with special characters removed and spaces replaced by underscores.

After that, `transform_data.py` reads all text files from HDFS `/data` using `wholeTextFiles()`, which returns (filename, content) pairs. I parse the filename to get the doc_id and title. I also clean the text (remove tabs and newlines) and then write it as a single tab-separated file to `/input/data` using `coalesce(1)`. The format is:

`doc_id\ttitle\ttext`

This single-partition file is the input for the MapReduce pipelines.

1.2 Indexing Pipeline

I built two MapReduce jobs using Hadoop Streaming with Python scripts.

Job 1: Document Statistics (`mapper1.py` / `reducer1.py`)

The mapper reads each tab-separated line, preprocesses the text (lowercase, remove non-alphanumeric characters), and counts words. It outputs `doc_id\ttitle\tdl` where `dl` is the document length.

The reducer just forwards all lines but also keeps track of total document length and count. At the end it writes a special line `__STATS__\tN\tavg_dl` with the corpus-level statistics. I use a single reducer (`-D mapreduce.job.reduces=1`) because the stats line needs to aggregate across all documents.

Job 2: Inverted Index (`mapper2.py` / `reducer2.py`)

The mapper tokenizes each document (same preprocessing as Job 1) and counts term frequencies with Python's `Counter`. It outputs `term\tdoc_id\ttf` for each unique term.

The reducer collects all (doc_id, tf) pairs for each term (Hadoop already sorts by key) and outputs `term\tdf\tdoc_id1:tf1,doc_id2:tf2,...` where `df` is the number of documents containing that term.

The preprocessing is the same in both mappers and in the query application: 1. Convert to lowercase 2. Remove all non-alphanumeric characters except spaces 3. Split on whitespace

1.3 Cassandra Storage

I store the index in 4 Cassandra tables:

Table	Primary Key	Columns	Purpose
`documents`	`doc_id`	title, dl	Document metadata and length

Table	Primary Key	Columns	Purpose
`vocabulary`	`term`	df	Document frequency per term
`inverted_index`	`(term, doc_id)`	tf	Term frequency per document
`corpus_stats`	`id`	num_docs, avg_dl	Corpus-level BM25 parameters

I chose this schema because BM25 queries need to look up data by term. The `inverted_index` table has a compound primary key `(term, doc_id)` so I can get all postings for a term in one query.

The `store_index_helper.py` script reads MapReduce output from HDFS and inserts rows into Cassandra using prepared statements.

1.4 BM25 Ranking

The query application (`query.py`) uses PySpark RDD to compute BM25 scores. The formula:

$$BM25(q,d) = \sum_{t \in q} \log \frac{N}{df(t)} \cdot \frac{(k_1+1) \cdot tf(t,d)}{k_1 \cdot \left[(1-b) + b \cdot \frac{dl(d)}{avg_dl}\right] + tf(t,d)}$$

I use k1 = 1.5 and b = 0.75.

For each query, I first preprocess it the same way as during indexing. Then I fetch the needed data from Cassandra on the driver side: corpus stats, vocabulary, postings and document info. I create an RDD from the postings and map each one to a (doc_id, partial_score) pair using BM25. Then I use `reduceByKey` to sum scores per document and take the top 10.

I broadcast the lookup data to workers so they don't query Cassandra themselves.

1.5 Incremental Indexing

The `add_to_index.sh` script takes a text file, gets the doc_id and title from the filename, and tokenizes the content. It inserts the document into `documents`, then goes through each term and adds it to `inverted_index` and updates `df` in `vocabulary`. It also recalculates `avg_dl` and `num_docs` in `corpus_stats`.

I think this is simpler than re-running the full MapReduce for one document.

2. Demonstration

2.1 How to Run

```
docker compose up
```

This starts 3 containers: Hadoop master, slave node, and Cassandra. The `app.sh` script runs automatically and goes through the full pipeline.

2.2 Indexing

I exported the container logs to a text file because the terminal output is long and the scroll limit cuts earlier parts.

MapReduce Job 1 completed and produced document statistics for 1178 documents. The `__STATS__` line shows N=1178 and avg_dl=557.11.

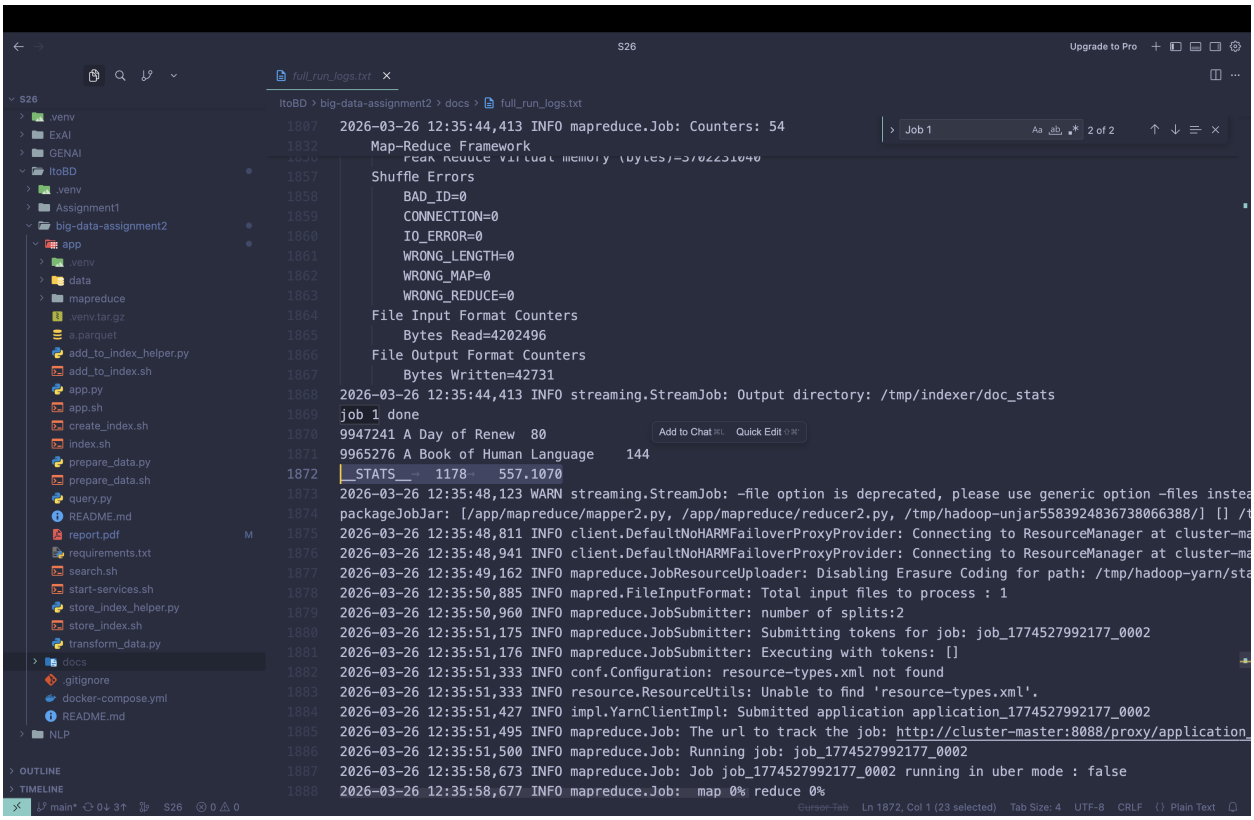


Figure 1: MapReduce Job 1 output

MapReduce Job 2 built the inverted index with 50,753 unique terms. After that, the Cassandra loading script created all tables and inserted the data.

2.3 Search Queries

I ran 3 queries to test the search engine.

Query: "world war history"

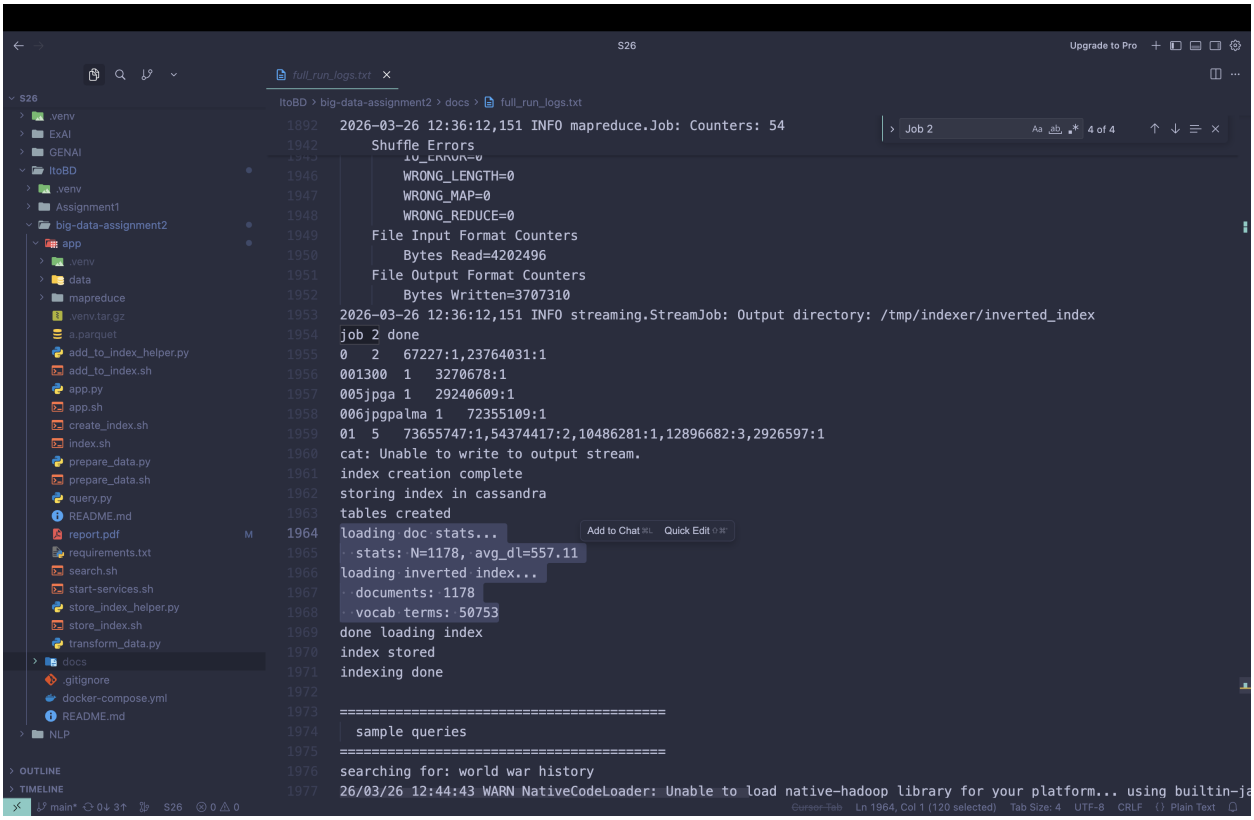
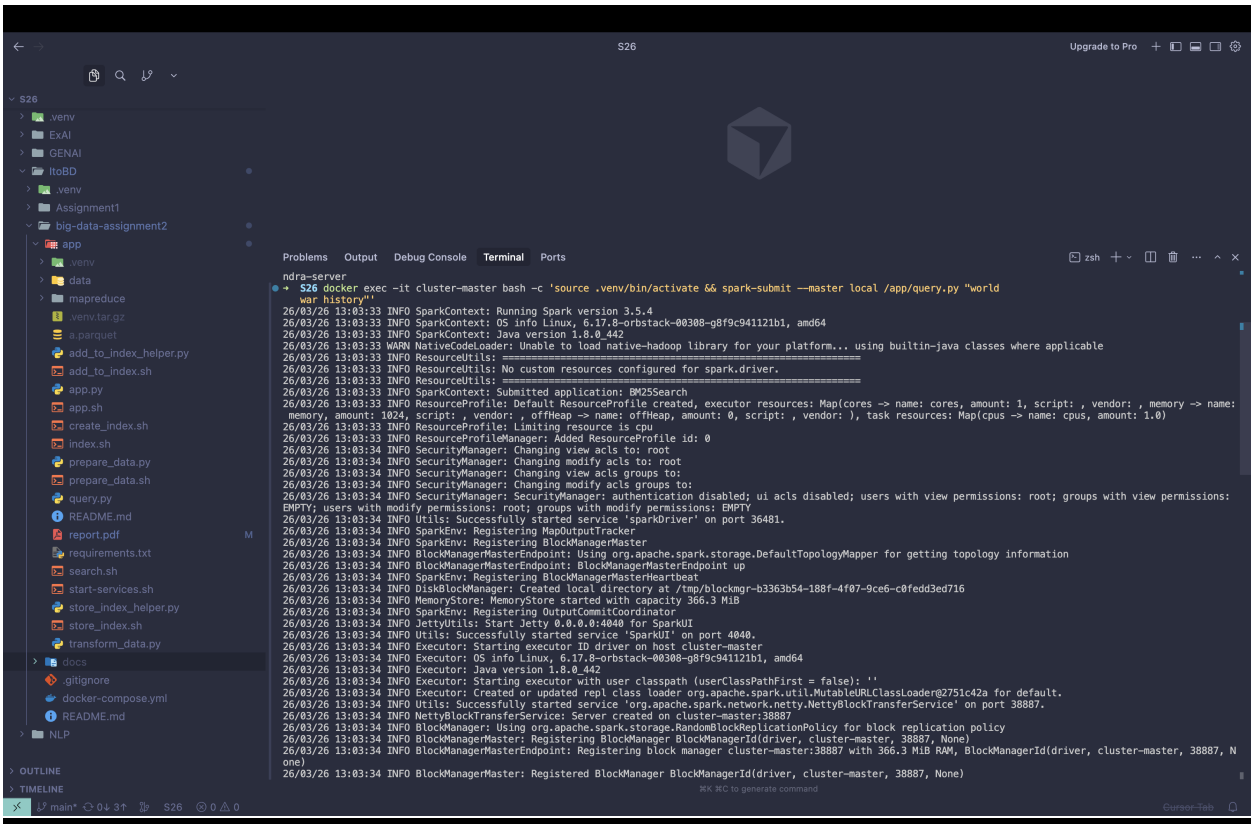
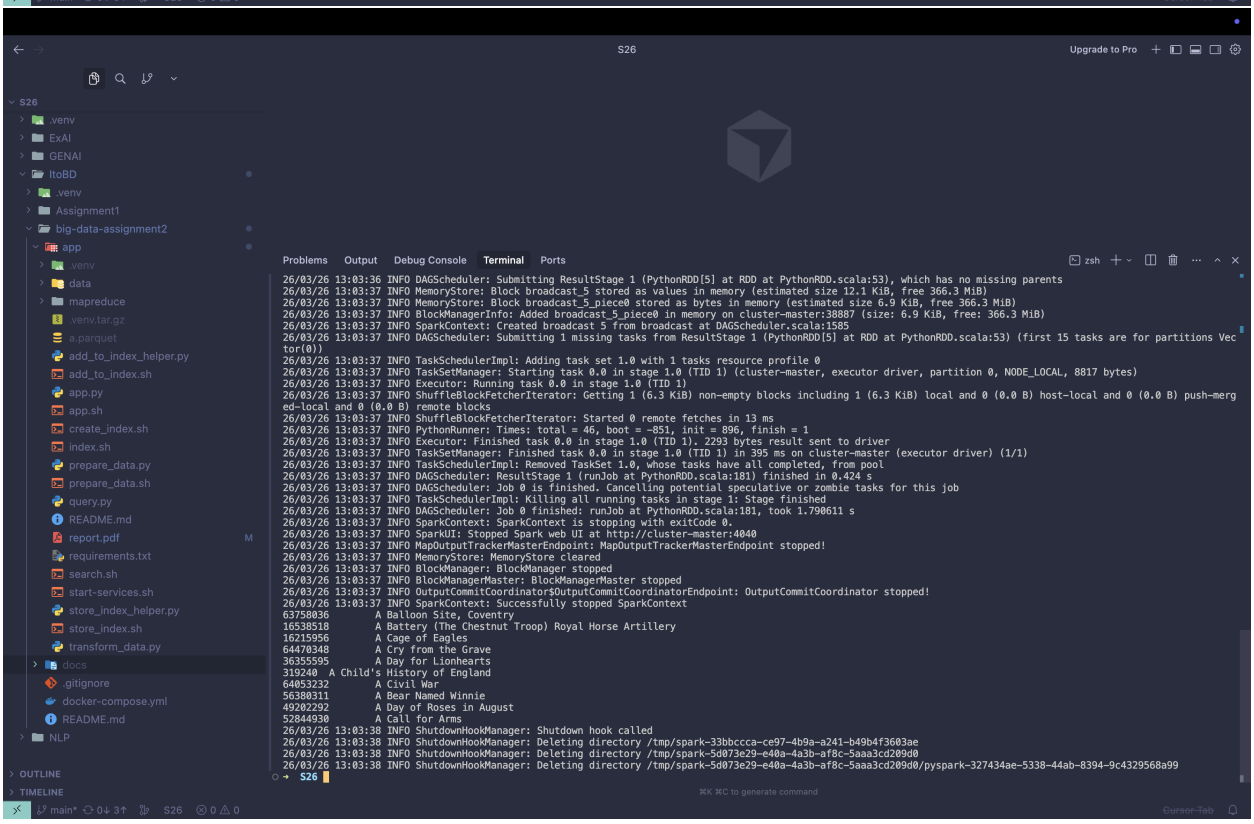


Figure 2: Job 2 + Cassandra loading

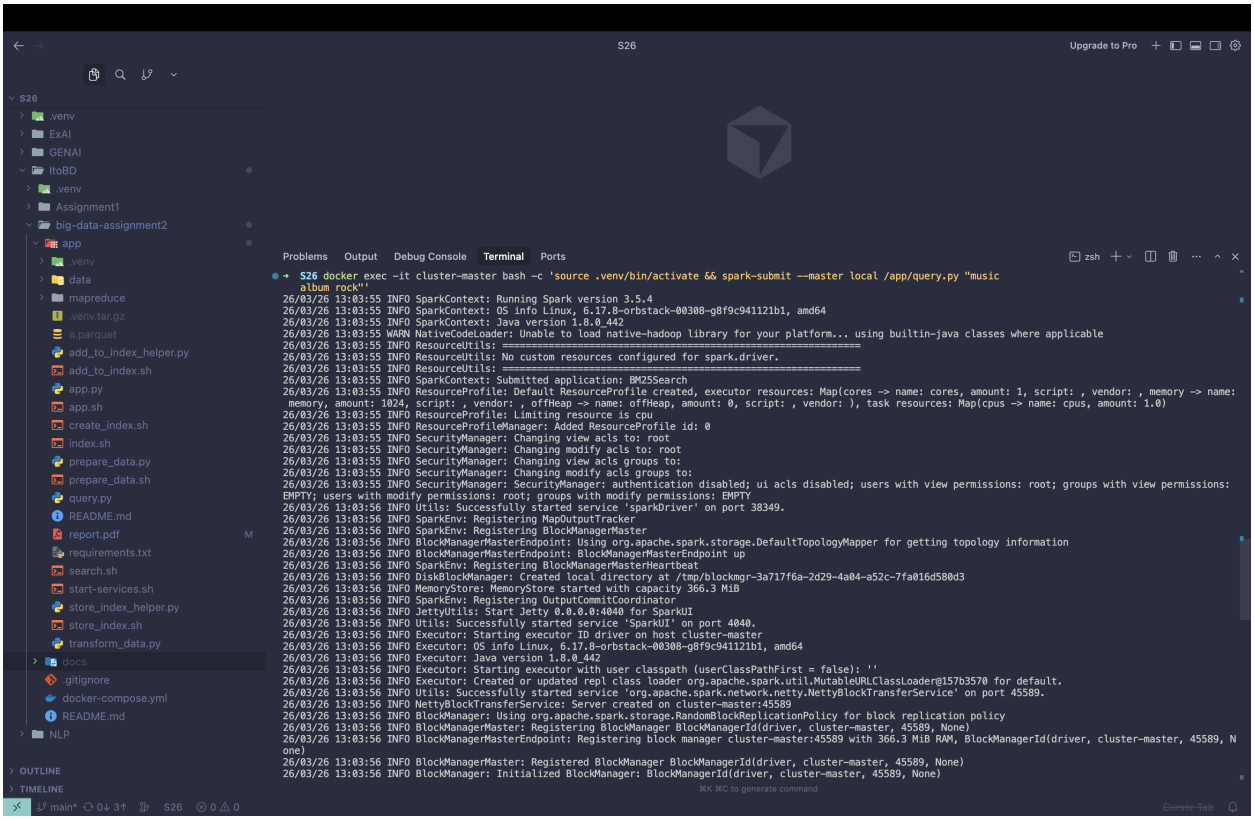


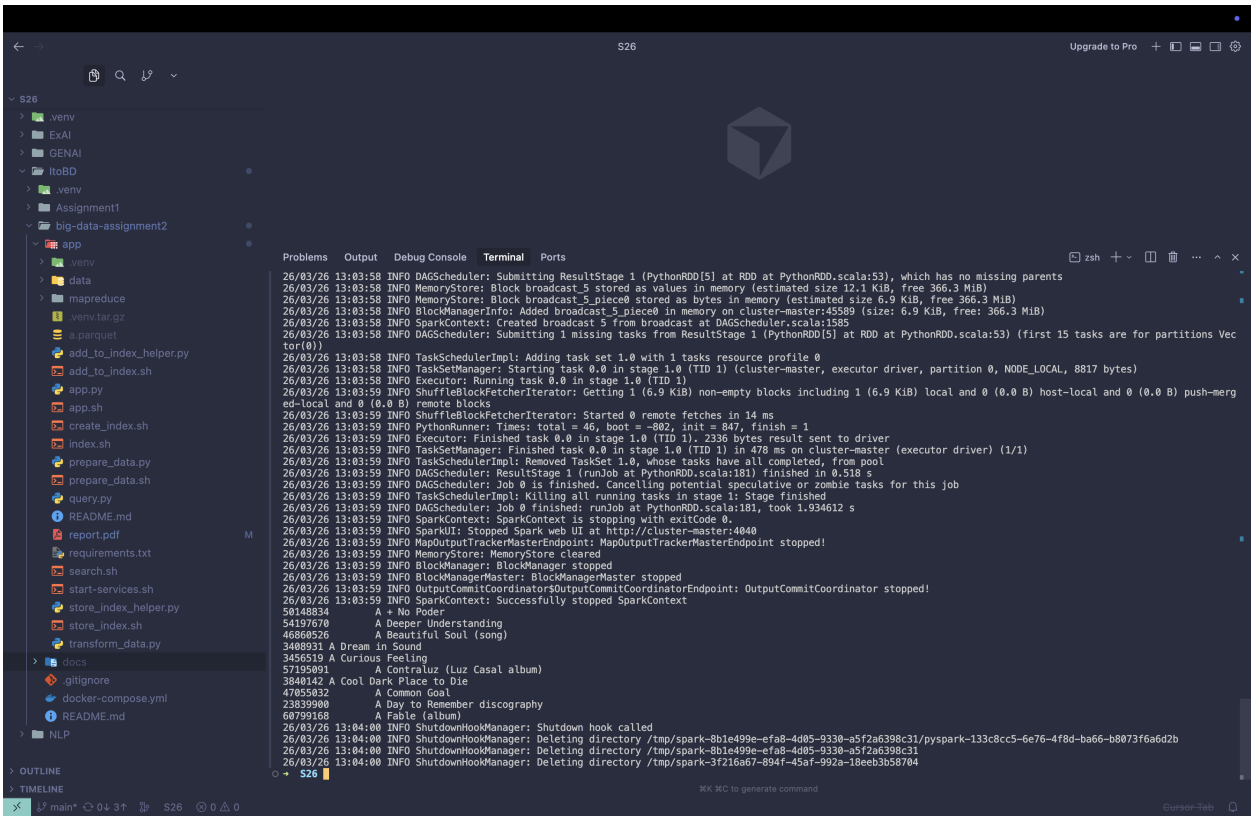


Top results: “A Battery (The Chestnut Troop) Royal Horse Artillery”, “A Day for Lionhearts”, “A Child’s History of England”, “A Civil War”. These are articles about war and military history,

which makes sense since they mention all three query terms.

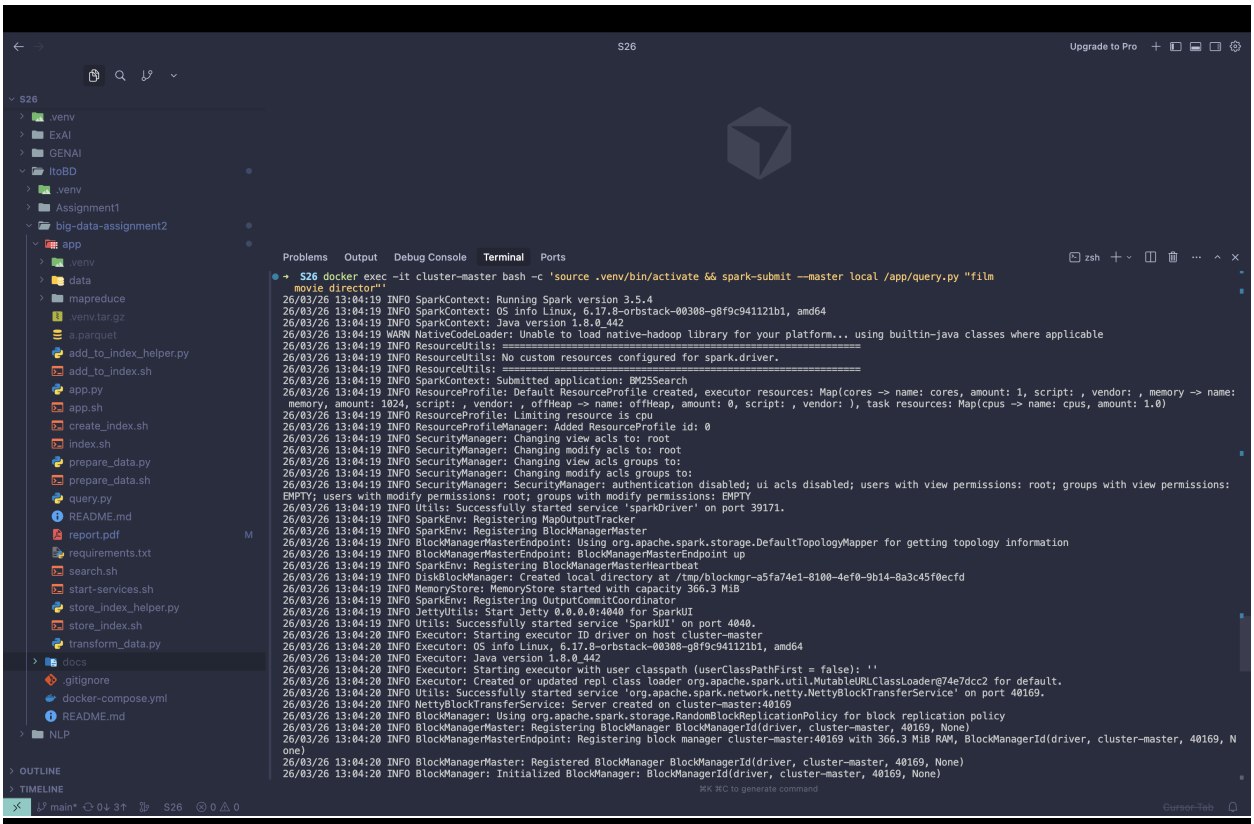
Query: “music album rock”

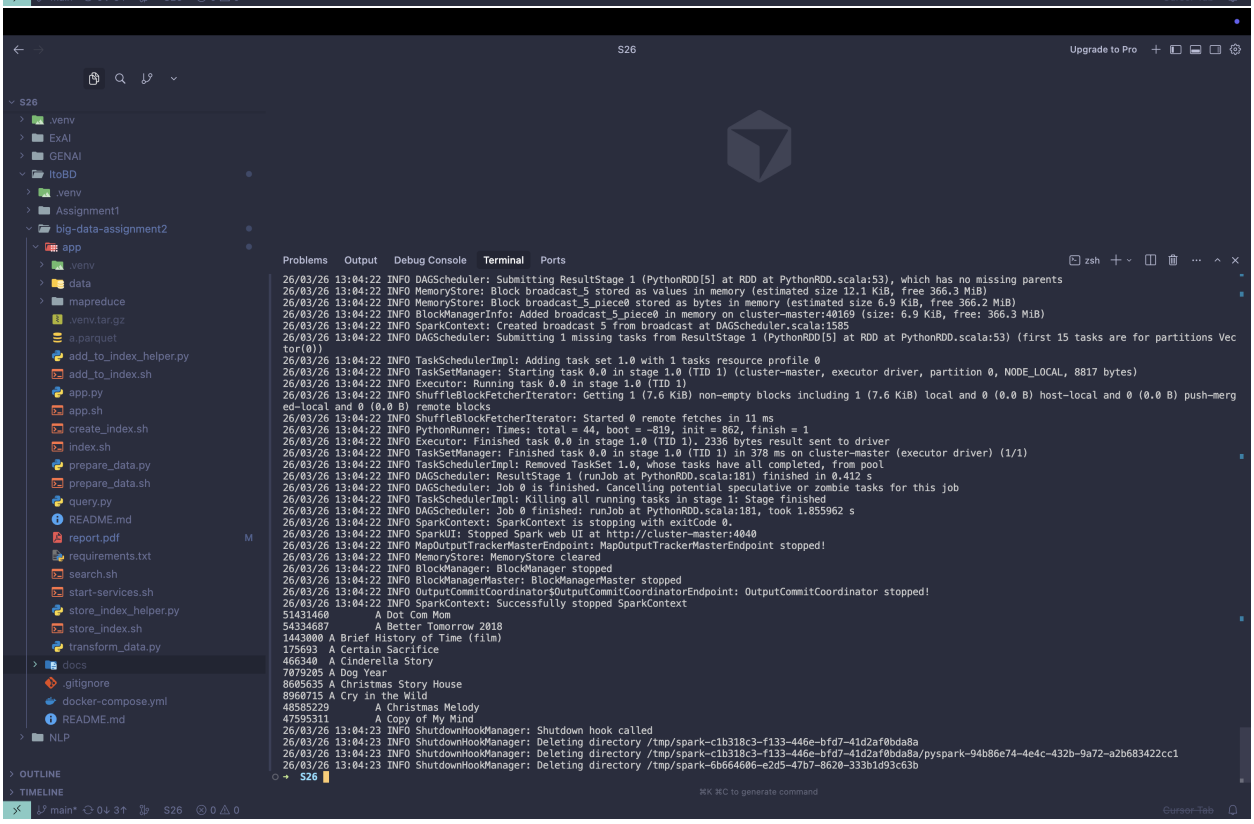




Top results include “A Deeper Understanding”, “A Beautiful Soul (song)”, “A Curious Feeling”. All music-related articles. So the engine finds the right kind of documents.

Query: “film movie director”





Same pattern here, top results are all film articles: “A Better Tomorrow 2018”, “A Brief History of Time (film)”, “A Cinderella Story”, “A Christmas Story House”. Articles that mention all three

terms get higher scores.

3. Known Issues

I developed on a MacBook with Apple Silicon (ARM architecture). The Docker image `firasj/spark-docker-cluster` is built for x86_64, so it runs under emulation on my machine. The MapReduce streaming jobs and Cassandra worked fine under emulation. But Spark on YARN was inconsistent. Sometimes the queries completed on YARN, other times the Application Master got stuck in ACCEPTED state and never registered with the ResourceManager.

I had a successful run where all 3 search queries worked on YARN, but I didn't save screenshots at that point. After that I couldn't reproduce it reliably. I searched online and found that some Apple Silicon users report similar problems with Spark's YARN integration under x86 emulation.

For the demonstration screenshots I ran the queries with `--master local` using `docker exec` to show the results reliably. The `search.sh` script in the repository uses `--master yarn` as required by the assignment. On an x86 machine the full pipeline works end-to-end without issues.